Assignment 4, explained

California Housing — from split to Random Forest

A plain-language walkthrough of every question,
with the numbers from the solution notebook.

SVNIT · Department of Artificial Intelligence
Machine Learning · u24ai028

The whole assignment in one page

You are given about 20,640 districts from the 1990 California census. Each row is one district, not one house. The job is to predict median_house_value from location, age, rooms, population, income, and how close the district is to the ocean.

The work has three stages. First, split the data so the test set looks like the real world. Second, clean the columns (fill missing numbers, scale them, turn ocean_proximity into 0/1 columns) and try three models. Third, tune the best model, score it once on the test set, and also solve a tiny linear regression by hand with the Normal Equation.

Columns

- longitude, latitude — where the district is
- housing_median_age — typical age of houses there
- total_rooms, total_bedrooms, population, households — size of the district
- median_income — typical income, in units of $10,000
- ocean_proximity — text: <1H OCEAN, INLAND, ISLAND, NEAR BAY, NEAR OCEAN
- median_house_value — the number you predict

One warning you will see in the plots: house values are capped at $500,001. Anything more expensive was recorded as that number. The model cannot learn prices above the cap.

Q1. Split by income, not at random

A random 80/20 split can, by luck, put too many rich districts in the test set. Income is the strongest clue about price, so the test set must have the same mix of incomes as the full data.

median_income is a continuous number, and stratified splitting needs groups. So you cut it into 5 bins called income_cat:

bins: 0 – 1.5 – 3.0 – 4.5 – 6.0 – infinity labels: 1, 2, 3, 4, 5

StratifiedShuffleSplit(test_size=0.2, random_state=42) then keeps each bin's share almost identical in train and test.

income_cat	Train count	Test count	Share
1 (poorest)	657	165	about 4%
2	5,265	1,316	about 32%
3	5,789	1,447	about 35%
4	2,911	728	about 18%
5 (richest)	1,890	472	about 11%

Train has 16,512 districts. Test has 4,128. After the split, income_cat is dropped. It was only a tool for splitting. The model still sees the original median_income.

Say this: I stratified on income bins so the test set has the same rich/poor mix as the full data. A plain random split might not.

Q2. One pipeline, two kinds of columns

Models need numbers, and they need every row filled in. total_bedrooms is missing for 158 training districts. ocean_proximity is text. A ColumnTransformer sends each kind of column down its own

path.

Numeric path (8 columns)

- SimpleImputer(strategy="median") fills blanks with the median. The median is not pulled around by a few huge districts.
- StandardScaler subtracts the mean and divides by the standard deviation, so every column is on a similar scale.

Category path (ocean_proximity)

- OneHotEncoder makes 5 columns, one per category. A district near the bay is [0, 0, 0, 1, 0], not the number 3.
- handle_unknown="ignore" means a brand-new category at test time becomes all zeros instead of crashing. ISLAND is very rare, so this matters.

Resulting training array: shape (16512, 13). That is 16,512 rows and 13 columns = 8 scaled numbers + 5 one-hot columns.

fit_transform is used on the training set only. The test set gets transform. The median and the mean/std must be learned from training rows. If you compute them on the test set too, the test set has leaked into the preprocessing.

Say this: 13 columns: 8 scaled numeric features plus 5 yes/no ocean categories. I fit the imputer and scaler on train only.

Q3. Three models, scored with 5-fold CV

Cross-validation cuts the training set into 5 folds. Each model is trained on 4 folds and scored on the leftover fold, five times. The reported number is the average RMSE, plus how much that score wobbles (the std).

RMSE is the typical prediction error in dollars. Lower is better. sklearn returns the negative of RMSE because it always tries to maximise a score. The notebook flips the sign back.

Model	CV RMSE
Linear regression	$69,218 ± $690
Decision tree	$70,629 ± $1,448
Random forest (100 trees)	$49,942 ± $761

- Linear regression draws one flat relationship. Price depends on location and income in curves and jumps, so a straight fit leaves about $69k of error.
- One decision tree can memorise the training districts. On a new fold it is worse than the straight line, and its score jumps around (largest std).
- A random forest averages many trees. Each tree sees a random sample of rows and a random subset of columns. Averaging cancels the wild mistakes. It wins, at about $50k typical error.

Say this: The forest wins because averaging trees cuts the overfitting that kills a single tree, and the splits capture patterns a straight line misses.

Q4. Grid search on the forest

Two knobs are tried. n_estimators is how many trees (50, 100, or 200). max_features is how many columns a split is allowed to look at (2, 4, 6, or 8). That is 3 × 4 = 12 combinations. Each one is

scored with 5-fold CV, so the search trains 60 forests.

Best combination: n_estimators = 200, max_features = 8. Best CV RMSE = $49,015.

More trees make the average steadier. The gain from 100 to 200 trees is only about $140, so extra trees help a little and then flatten out. max_features = 2 makes each tree too weak (RMSE around $51–52k). Allowing 8 features at each split was best inside this grid.

200 and 8 are both at the edge of the grid. The true best setting might be a bit higher. A follow-up search would try max_features of 8, 10, and 12.

Say this: GridSearchCV tried every pair of tree-count and feature-count, kept the one with the lowest cross-validated RMSE, then refit that forest on all of the training data.

Q5. One honest score on the test set

The tuned forest is used once on the 4,128 districts it has never seen. Mean test house value is about $206,258.

Metric	Value	In words
Test RMSE	$46,564	typical error, 22.6% of the mean price
Test MAE	$30,806	average absolute error, 14.9% of the mean
Test R^2	0.8336	83% of the price variation is explained

RMSE is larger than MAE because squaring punishes the big misses. Those big misses sit near the $500,001 cap, where the true price was chopped off. Test RMSE ($46.6k) is close to the CV RMSE ($49.0k), so the tuning did not cheat by fitting the test set.

Say this: On new districts the forest is typically about $47k off, and it explains 83% of the variation in price.

Q6. The Normal Equation, with only NumPy

Linear regression has a closed-form answer. You do not have to search. With a column of 1s added for the intercept, the best weights are:

$$\theta = (X^T X)^{-1} X^T y$$

Here X has only two columns from the training set: median_income and housing_median_age, plus the column of 1s. The same two columns are given to sklearn's LinearRegression. The two answers match to many decimal places. The tiny percent difference (about 10^{-11} %) is rounding error. Both methods minimise the same squared error.

Parameter	Value	Meaning
$\theta 0$ intercept	−10,706.89	baseline value
$\theta 1$ median_income	43,041.08	per +1 income unit
$\theta 2$ housing_median_age	1,776.05	per extra year of age

$$\text{value} = -10{,}706.89 + 43{,}041.08 \times \text{income} + 1{,}776.05 \times \text{age}$$

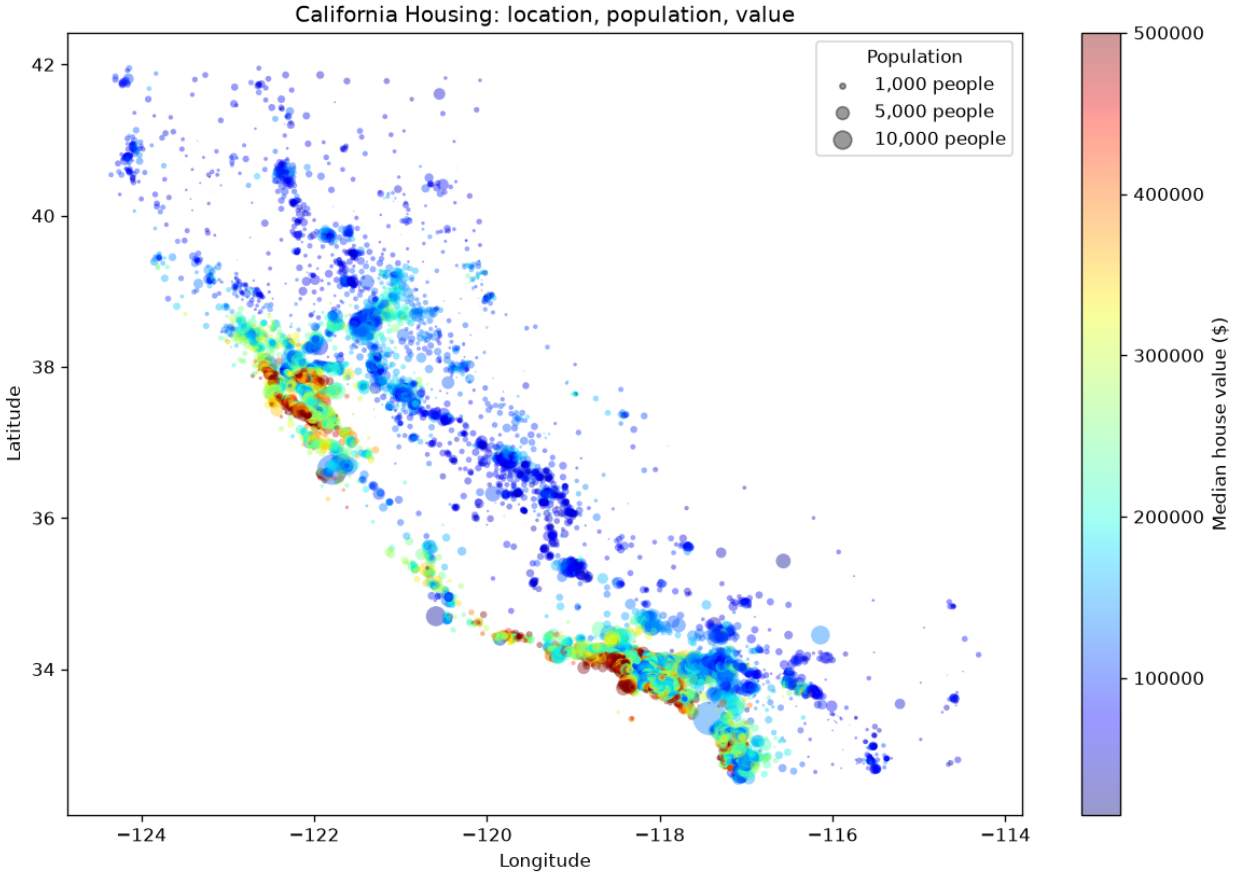
median_income is in units of $10,000. So a $10,000 rise in typical household income goes with about $43,000 more in median house value, if age stays the same.

The Normal Equation is the right tool when there are few columns. Inverting X^TX gets slow as the number of columns grows, and it fails if two columns are copies of each other. Then you use gradient descent instead. That is Assignment 5.

Say this: NumPy and sklearn solve the same least-squares problem, so θ matches. The income slope is about $43k of house value per income unit.

Q7. Where the expensive districts are

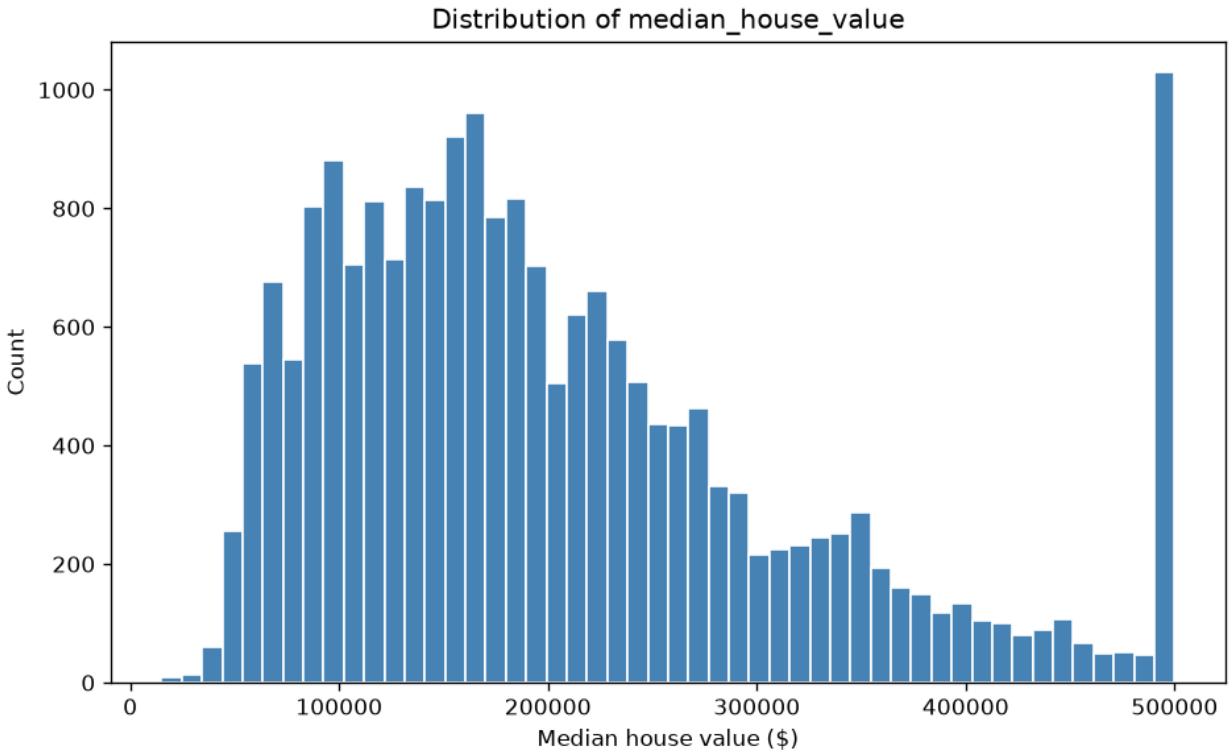
Each point is one district. Horizontal position is longitude, vertical is latitude, so the cloud is the shape of California. Bigger dots mean more people (dot size = population / 100). Colour is median house value. alpha=0.4 lets overlapping dots show through.



Coast and bay districts (Bay Area, Los Angeles, San Diego) are the expensive ones. Inland districts are cheaper. Location should matter in the model.

Q8. The shape of house values

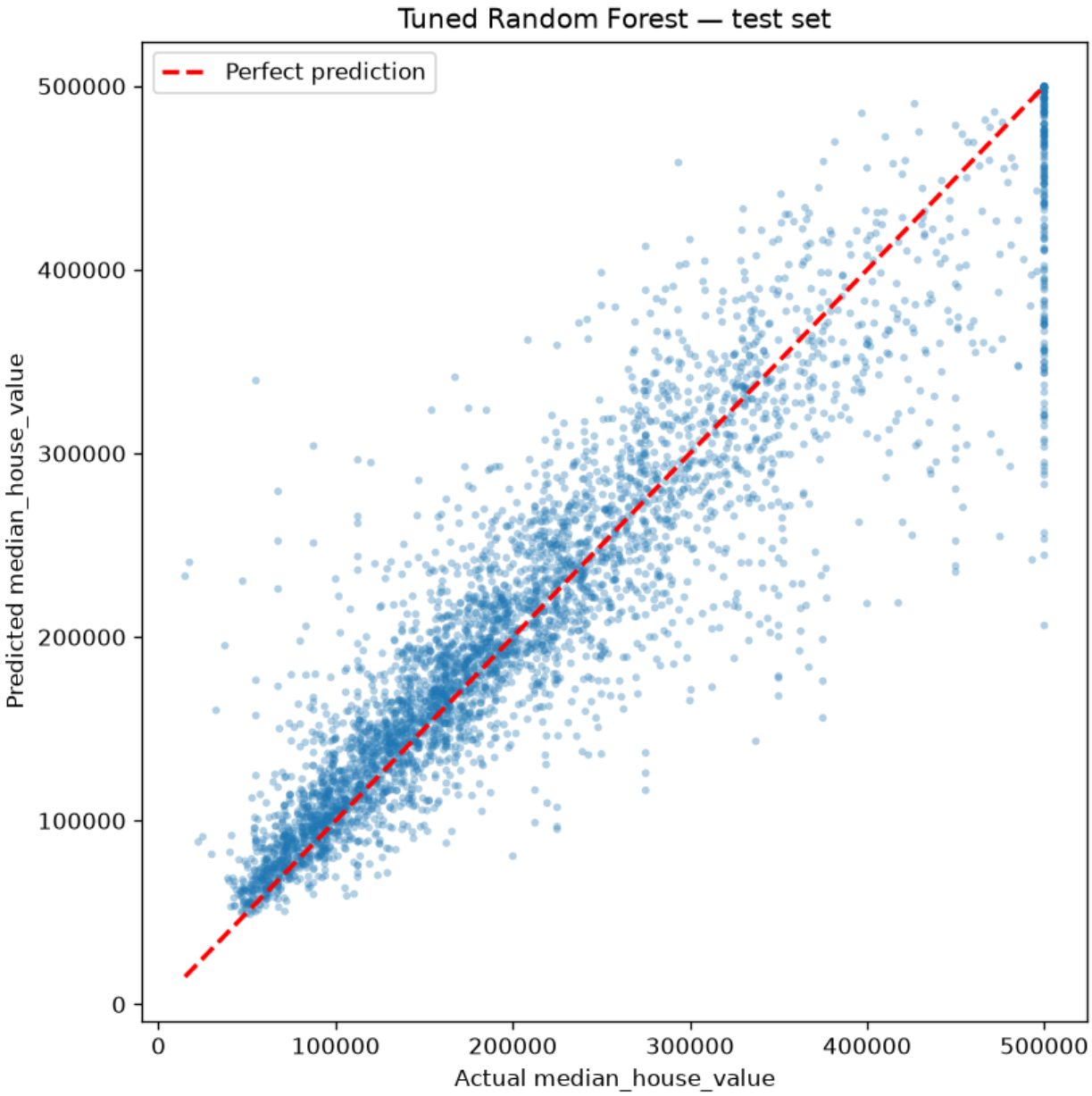
Mean value is $206,856. Median is $179,700. The mean is higher because a long right tail of expensive districts pulls it up. Skewness is 0.98, so the distribution is right-skewed. The tall bar at the far right is the $500,001 cap, not a real pile of houses that all cost exactly that.



Most districts sit between about $100k and $250k. The spike at $500,001 is the census cap.

Q9. Predicted vs actual

Each point is one test district. The horizontal axis is the real median value. The vertical axis is the forest's prediction. The red dashed line is perfect prediction: predicted = actual. Points near that line are good. A vertical strip at $500,001 is the cap: those districts were worth more, but the data says 500001, so the forest predicts below the line.



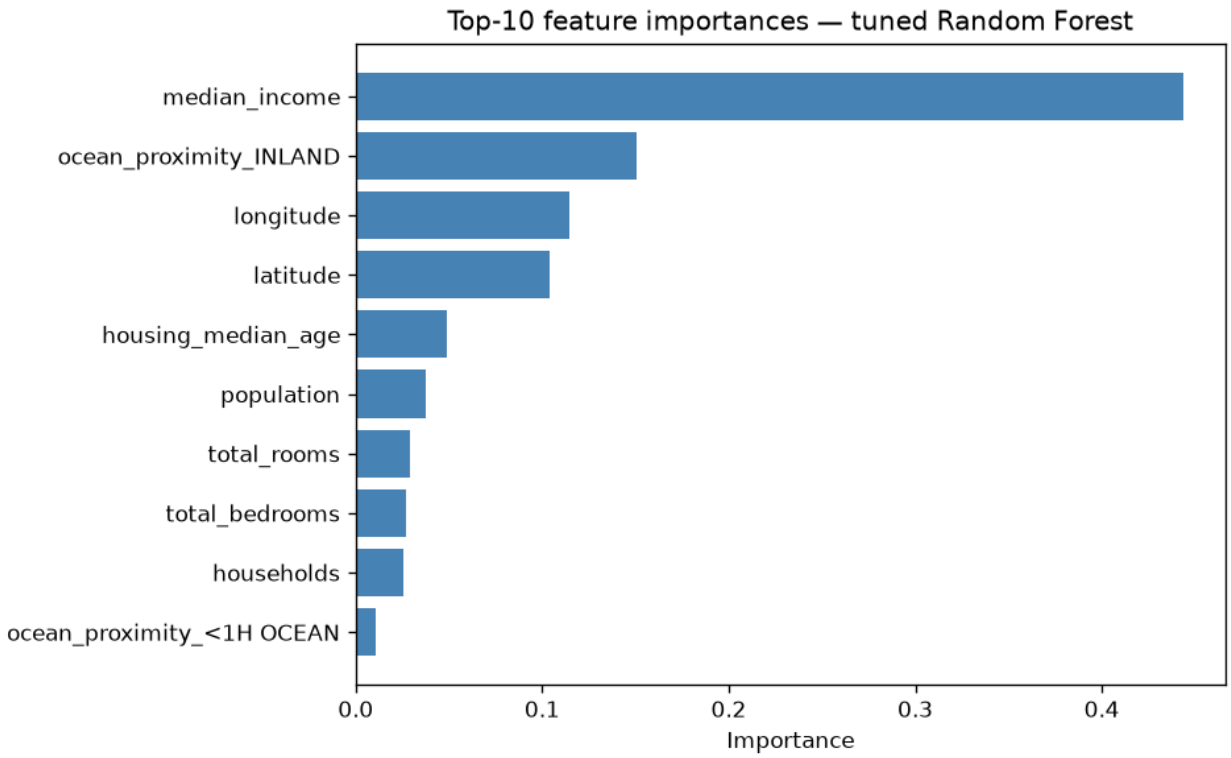
Tight to the diagonal in the middle. The big misses are the capped expensive districts.

Q10. Which columns the forest uses

Feature importance is how much a column reduces error across all the splits in all the trees. The numbers add up to 1. They say "this column was useful," not "this column causes higher prices."

Rank	Feature	Share
1	median_income	44.4%
2	ocean_proximity = INLAND	15.1%
3	longitude	11.4%
4	latitude	10.4%
5	housing_median_age	4.9%

Income plus location (inland flag, longitude, latitude) is almost the whole model. The top 10 features hold 99% of the importance. That matches the map: coast vs inland, and richer districts.



median_income alone is 44% of the forest's importance.

Say this: The forest mostly uses income and location. Inland districts get their own strong signal, which matches the map.

Short answers if you are asked

- Why stratify? So the test set has the same income mix as the full data.
- Why median impute? It is not dragged by extreme districts.
- Why scale? Rooms are in the thousands, income is around 1–15. Linear models care about that.
- Why one-hot, not 0,1,2,3? Those numbers would pretend INLAND < NEAR BAY.
- Why is the tree worse than linear regression? It memorises and then fails on new folds.
- Why does the forest win? Many trees, averaged, keep the useful splits and cancel the noise.
- What is RMSE vs MAE? Both are in dollars. RMSE punishes large errors more.
- What is $R^2 = 0.83$? 83% of the variation in price is explained.
- Why do NumPy and sklearn match in Q6? They minimise the same squared error.
- What is wrong with this data? It is from 1990, values are capped at $500,001, and each row is a district average.

Assignment 5, explained

Gradient descent, polynomials, and regularisation

A plain-language walkthrough of every question,
with the numbers from the solution notebook.

SVNIT · Department of Artificial Intelligence
Machine Learning · u24ai028

The whole assignment in one page

Assignment 4 used sklearn to fit models. This assignment asks you to see how a model actually moves toward the answer, and what happens when the model is too simple or too flexible.

- Q1–Q3 use made-up data: y = 4 + 3x + noise. You walk θ toward the best line three ways: batch, stochastic, and mini-batch gradient descent.
- Q4 uses a curve, y = 0.5 x^2 + x + 2 + noise. A degree-1 line underfits. A degree-300 polynomial overfits. Degree 2 matches the truth.
- Q5–Q6 use real red-wine data. Ridge, Lasso, and Elastic Net predict the quality score. Then logistic regression predicts "good wine" (score 7 or more).
- Q7 uses the Iris flowers. Logistic regression draws the boundary between three species from petal length and petal width.

Random draws use seed 42, so a re-run gives the same θ.

The one idea behind Q1–Q3

A line is y = θ0 + θ1 x. θ0 is the intercept, θ1 is the slope. The error of a guess is the mean squared error:

MSE = average of (prediction − actual)²

Gradient descent starts at a random θ and repeatedly steps downhill on that error. The gradient is the slope of the error. The learning rate η says how big the step is.

θ_new = θ_old − η × gradient

The data is X = 2 × random(100, 1) and y = 4 + 3X + noise, so the true line is θ = [4, 3]. Noise means the best fit on these 100 points is a bit off that. The Normal Equation on the same points gives θ0 = 4.2151 and θ1 = 2.7701, with MSE = 0.8066. That is the target the gradient methods are walking toward. The starting θ was random: about [0.01, 1.45].

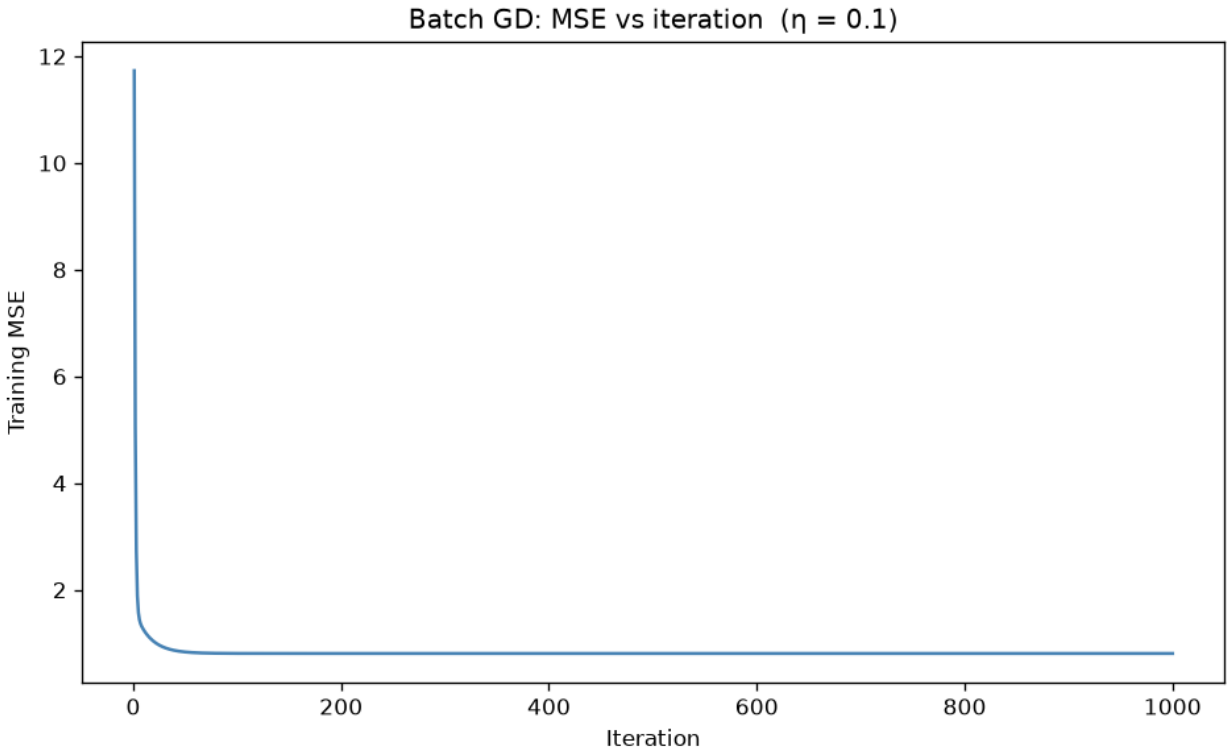
Q1. Batch gradient descent

Every step uses all 100 rows at once. Learning rate is 0.1. There are 1,000 iterations, so 1,000 parameter updates. The gradient of the MSE is:

gradient = (2 / m) X^T (Xθ − y)

The 2 appears because the derivative of (error)² brings down a 2. m is 100. A column of 1s is glued onto X so θ0 has something to multiply.

After 1,000 steps, batch GD lands on θ0 = 4.2151 and θ1 = 2.7701. That is the same answer as the Normal Equation, so it has fully converged. Training MSE ends at 0.8066. The MSE drops below 1.0 at epoch 22. One epoch here is one pass over the data, which for batch GD is one iteration.



The error falls almost immediately, then the line is flat. Extra iterations after that do not change θ.

Say this: Batch GD uses every row on every step. With η = 0.1 it reaches the least-squares line, the same θ as the Normal Equation.

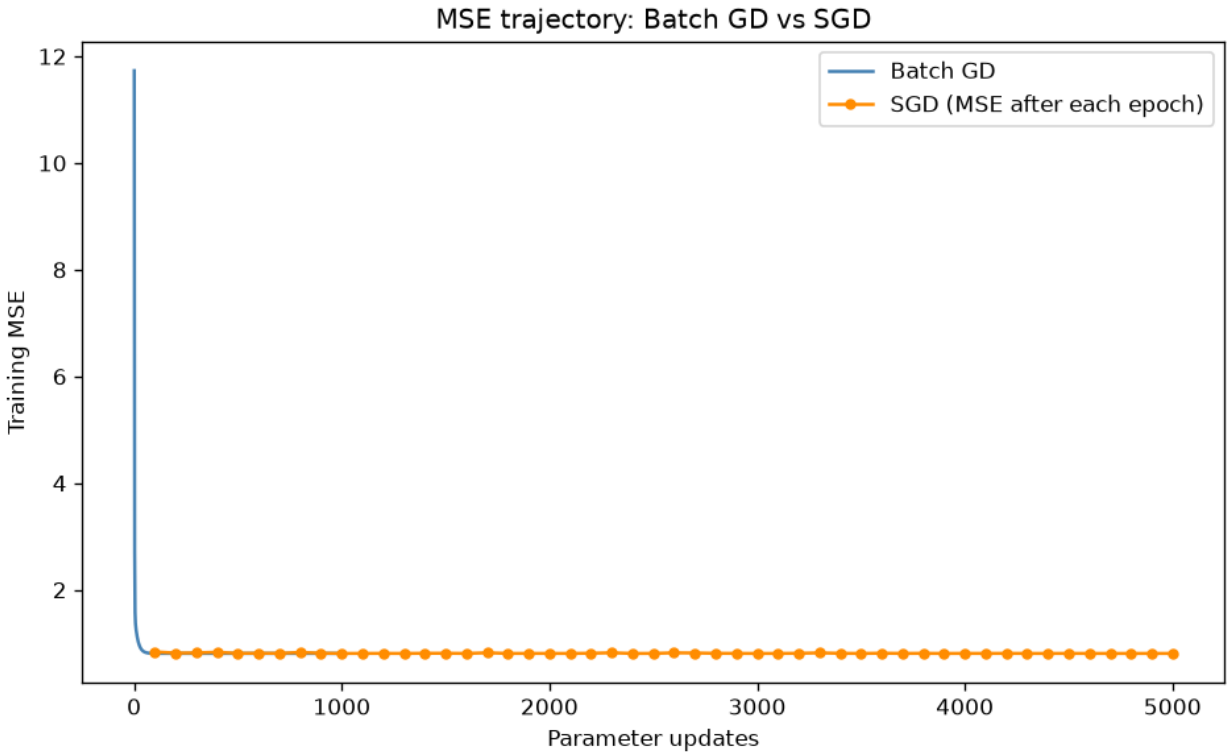
Q2. Stochastic gradient descent

Stochastic means random. Each step uses one random row, not all 100. The step size shrinks over time so the path can settle:

$$\eta = t0 / (t + t1) \quad \text{with } t0 = 5, t1 = 50$$

t starts at 0 and goes up by 1 on every row. Early on, η = 5/50 = 0.1, the same size as batch GD. Later the steps get smaller.

50 epochs × 100 rows = 5,000 updates. Batch GD only did 1,000. SGD's final θ is 4.2151 and 2.7535, and the final MSE is 0.8069. Almost the same place as batch GD, with a slightly different slope because the path is noisy. MSE is already under 1.0 after the first epoch, because that epoch contains 100 updates.



Blue is batch GD (1,000 updates). Orange dots are SGD, measured after each epoch. Both sit on the same low error. SGD just keeps taking small noisy steps out to 5,000 updates.

Say this: SGD updates once per row, 5,000 times in total. Batch GD updates once per full pass, 1,000 times. They finish at almost the same θ.

Q3. Mini-batch, and the comparison table

Mini-batch is the middle ground. Each step uses 20 rows (batch_size = 20), in random order. The question does not give a separate learning rate, so the notebook uses the same decaying schedule as SGD, η = 5 / (t + 50). 50 epochs × (100 / 20) = 250 updates.

Final mini-batch θ is 4.1349 and 2.8384. Final MSE is 0.8085. It crosses MSE < 1.0 at epoch 6.

Method	**θ0**	**θ1**	**Epochs to MSE<1**	**Updates**
Batch GD	4.2151	2.7701	22	1,000
Stochastic GD	4.2151	2.7535	1	5,000
Mini-batch GD	4.1349	2.8384	6	250

Read the "epochs" column carefully. An epoch is one pass over the 100 rows. SGD looks fastest in epochs only because each epoch hides 100 updates. Mini-batch does the fewest updates overall (250) and still lands near the same error. Batch GD takes the most epochs, but every step is exact, which is why it matches the Normal Equation digit for digit.

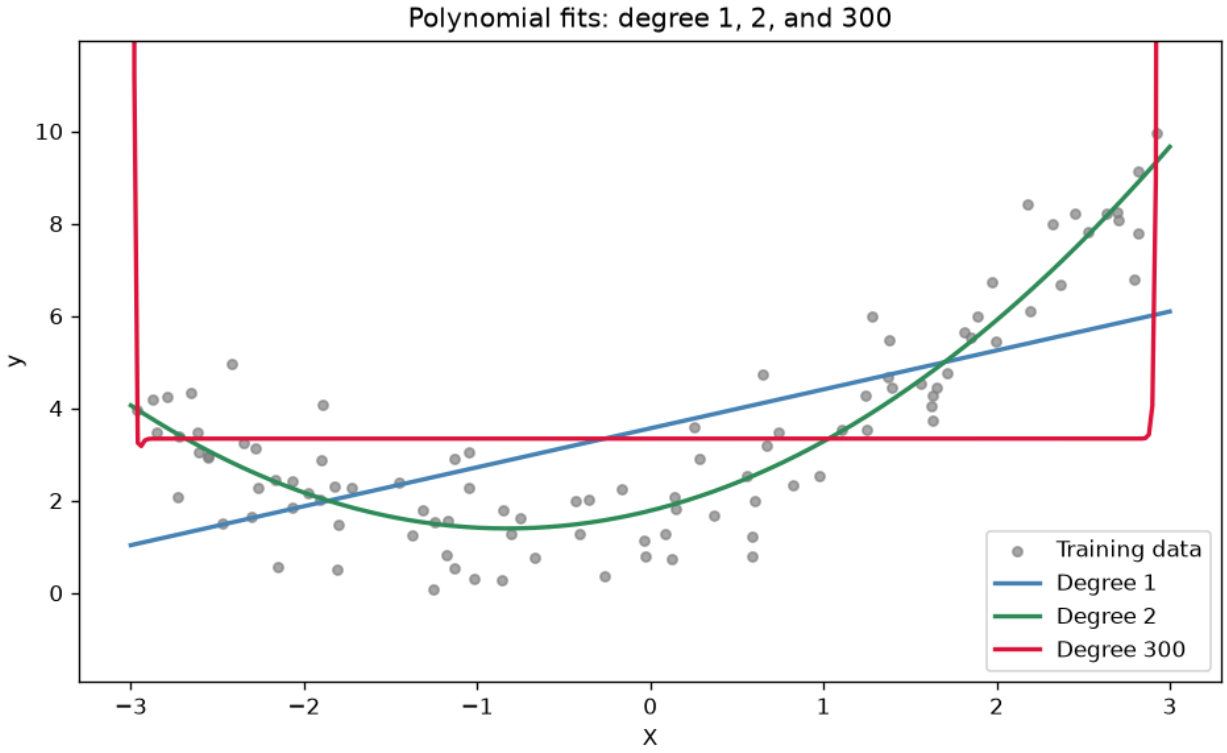
Say this: All three land near θ = [4.22, 2.77] and MSE ≈ 0.81. They differ in how many rows they use per step, and therefore in how many steps they take.

Q4. Too simple, just right, too flexible

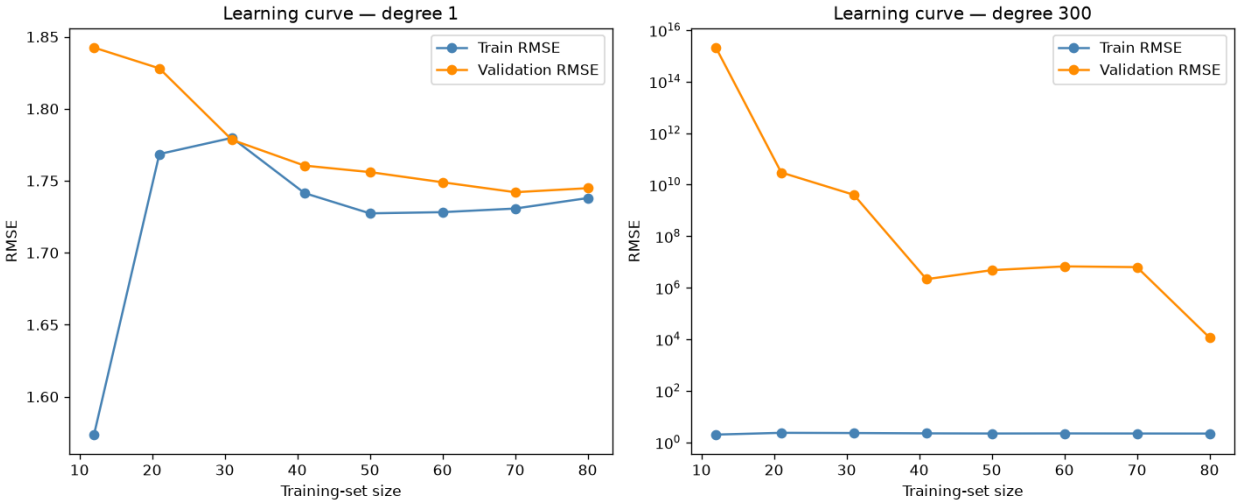
New data: 100 x-values spread from −3 to 3, and $y = 0.5\,x^2 + x + 2 +$ noise. The truth is a parabola. PolynomialFeatures builds columns x, x^2, x^3, … up to the degree you ask for. LinearRegression then fits a weight on each column.

Degree	Train RMSE	What you see
1	1.74	a straight line through a curve
2	0.88	the true shape, error ≈ the noise
300	2.20	a curve that shoots off at the edges

Degree 1 cannot bend, so it misses the dip in the middle. That is underfitting: both training and validation error stay high, and they stay close to each other (about 1.74). Degree 2 is the degree the data was built from, so the leftover error is basically the noise. Degree 300 has far more knobs than data points. On the learning curve its validation RMSE is about 11,500 while training RMSE is about 2.2. That gap is overfitting. The right-hand plot uses a log scale because the validation error is thousands of times larger than the training error.



Blue line underfits. Green curve follows the points. Red degree-300 curve is flat in the middle and explodes at $x = -3$ and $x = 3$.



Left: degree 1, train and validation error stay together and high. Right: degree 300, validation error (orange) stays far above training error.

Say this: Degree 1 underfits because a line cannot make a parabola. Degree 300 overfits because it has more coefficients than points, so it fails on new x values.

Q5. Ridge, Lasso, and Elastic Net on red wine

The file is winequality-red.csv, semicolon-separated, 1,599 wines and 11 chemical measurements. The target is quality, a score. The split is 80/20 with random_state=42: 1,279 train, 320 test. Every model is a pipeline: StandardScaler, then the regressor. Scaling matters because alpha penalises the size of the coefficients, and that penalty is only fair if the columns share a scale.

What the three penalties do

- Ridge adds α × (sum of squared coefficients). Coefficients get small, but they rarely become exactly 0. Every feature stays in the model.
- Lasso adds α × (sum of |coefficients|). Coefficients can become exactly 0, so Lasso deletes features.
- Elastic Net is both penalties at once. sklearn's default mix is l1_ratio = 0.5, half Lasso and half Ridge.

α is tried at 0.001, 0.01, 0.1, 1, and 10. Bigger α means a stronger penalty and a simpler model. The table is test RMSE. Lower is better.

Model	**0.001**	**0.01**	**0.1**	**1**	**10**
Ridge	0.6245	0.6245	0.6245	0.6245	0.6246
Lasso	0.6249	0.6269	0.6627	0.8107	0.8107
Elastic Net	0.6247	0.6263	0.6441	0.8107	0.8107

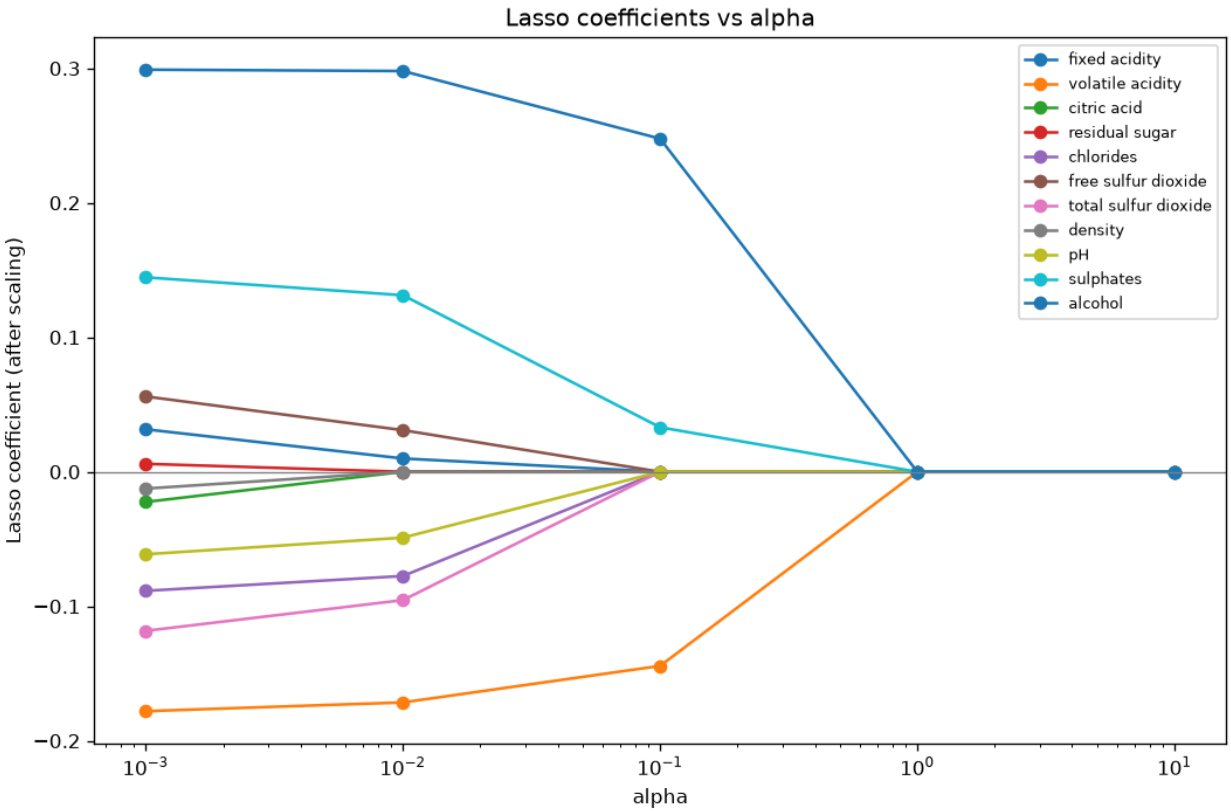
The best score in the table is Ridge at α = 0.001, RMSE 0.6245. Ridge barely changes as α grows, because shrinking coefficients a bit does not hurt this dataset. Lasso and Elastic Net get worse once α reaches 1. At α = 1 and α = 10 their RMSE is 0.8107, which is what you get from predicting the average quality for every wine: the penalty has wiped the model out.

Which Lasso coefficient hits zero first?

On the five required alphas, three features are already 0 at α = 0.01 and still nonzero at α = 0.001: citric acid, residual sugar, and density. A finer sweep between 0.001 and 0.01 separates them:

- residual sugar reaches 0 first, at about α = 0.0027
- density is next, at about α = 0.0033
- citric acid follows, at about α = 0.0057

alcohol, volatile acidity, and sulphates survive the longest. They only hit zero at α = 1. Those are the measurements the model most wants to keep.



Each line is one feature. As alpha grows, every line is pulled to 0. The lines that touch 0 earliest are the ones Lasso drops first.

Say this: Ridge is the most stable across alpha. Lasso starts deleting features as alpha grows, and residual sugar is the first coefficient to hit exactly zero.

Q6. Is the wine good? Logistic regression

quality is turned into two classes: good if the score is 7 or more, not good otherwise. There are 217 good wines and 1,382 not-good wines, so "good" is only about 14% of the data. The split is 80/20, stratified, so the test set keeps that 14%.

Logistic regression does not predict the class directly. It predicts a probability with the sigmoid function, and the usual cutoff is 0.5. Features are scaled first, same reason as Q5.

Metric	Score	Plain meaning
Accuracy	89.38%	overall fraction correct
Precision	69.57%	of wines called good, this fraction really is
Recall	37.21%	of truly good wines, this fraction was caught
F1	48.48%	balance of precision and recall

Accuracy looks strong because always saying "not good" would already be about 86% right. Recall of 37% means the model misses most of the good wines. That is the number to talk about, not the accuracy.

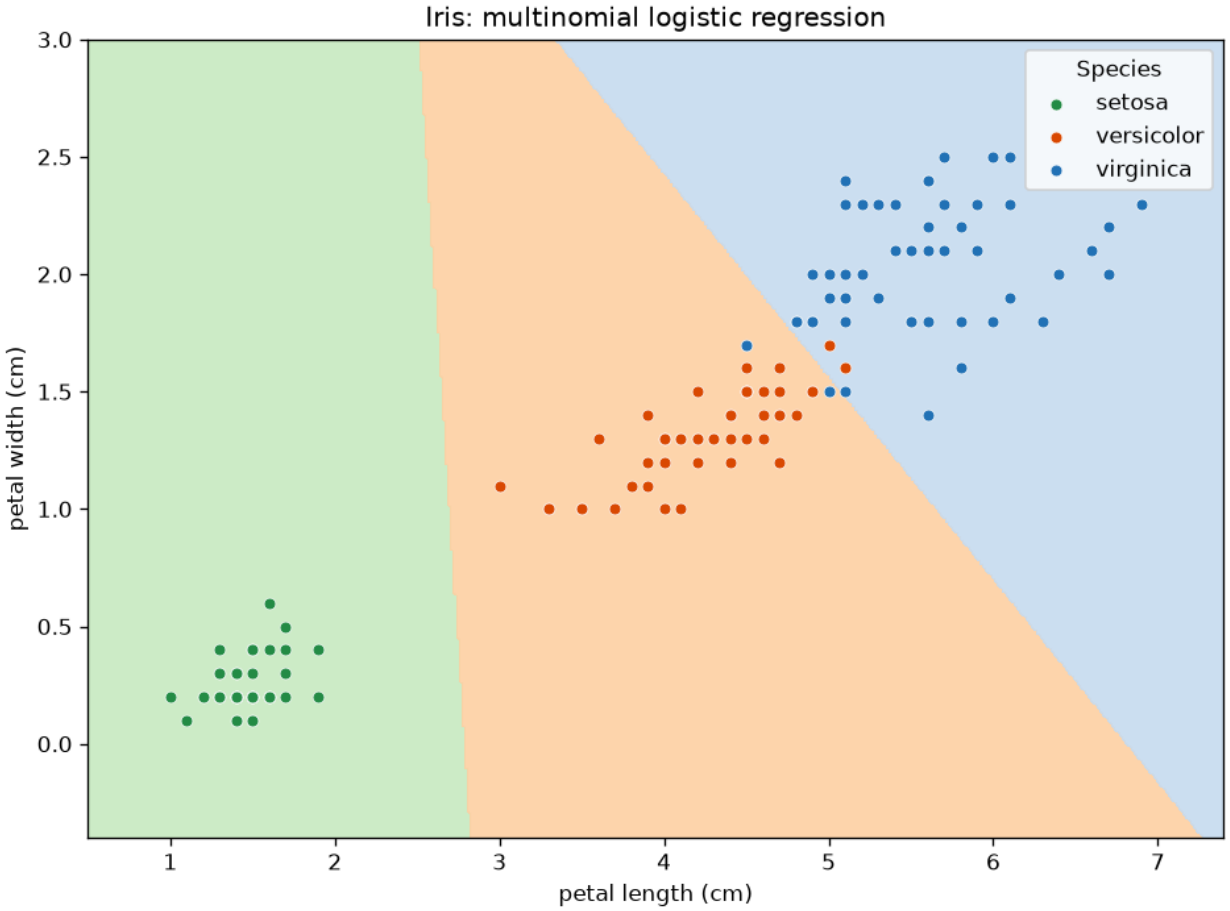
Say this: Accuracy is 89% partly because good wines are rare. Recall is only 37%, so most good wines are missed.

Q7. Iris: a picture of the decision boundary

Only two columns are used: petal length and petal width. There are three species: setosa, versicolor,

virginica. LogisticRegression on 3 classes fits the multinomial (softmax) model, which is what multi_class="multinomial" used to request. Current sklearn does that by default, and the old argument has been removed.

The filled colours are the model's prediction at every point in the plane. The boundary is where two class probabilities are equal. The dots are the 150 training flowers. Training accuracy is 96.67%. Setosa separates cleanly. Versicolor and virginica share a fuzzy border, which is where the few mistakes are.



Green is setosa, orange is versicolor, blue is virginica. Two petal measurements are enough to separate them.

Say this: With only petal length and petal width, multinomial logistic regression separates the three species at 96.7% training accuracy. Setosa is the easy one.

Short answers if you are asked

- What is a gradient? The slope of the error. We step in the opposite direction.
- What does the learning rate do? It sets the step size. Too big overshoots. Too small takes forever.
- Batch vs SGD vs mini-batch? All rows, one row, or a small group of rows, per update.
- Why does SGD's learning rate shrink? So the late steps stop bouncing around the minimum.
- Why does batch GD match the Normal Equation? Both minimise squared error, and 1,000 steps were enough to arrive.
- Underfitting vs overfitting? High error on both train and val, versus low train error and much higher val error.
- Why is degree 2 the right one? The data was built from a quadratic. Degree 1 cannot bend.

Degree 300 has more weights than points.

- Ridge vs Lasso? Ridge shrinks coefficients. Lasso can set them to exactly 0 and drop features.
- Why scale before Ridge/Lasso? Alpha penalises coefficient size. Unscaled columns would be penalised unfairly.
- Why is wine accuracy misleading? Only 14% of wines are "good", so a model can look accurate while missing most of them.

Assignment 6, explained

Fraud imbalance, hypothesis tests, and spam Naive Bayes

A plain-language walkthrough of both problems,
with the numbers from the solution notebook.

SVNIT · Department of Artificial Intelligence
Machine Learning · u24ai028

Two problems, one notebook

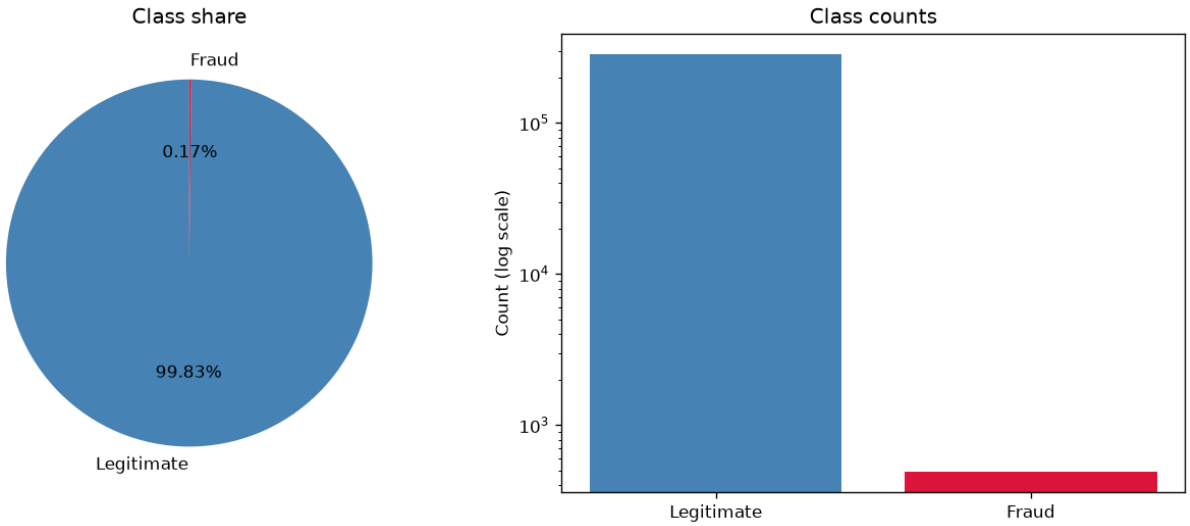
Problem 1 uses 284,807 card payments. 492 of them are fraud, which is 0.173%. The question is how to train a model when one class almost disappears, and how a hypothesis test can tell you which columns actually differ between the classes.

Problem 2 uses 5,572 SMS messages, labelled ham or spam. The text is cleaned, turned into word counts, and then two Naive Bayes models are compared.

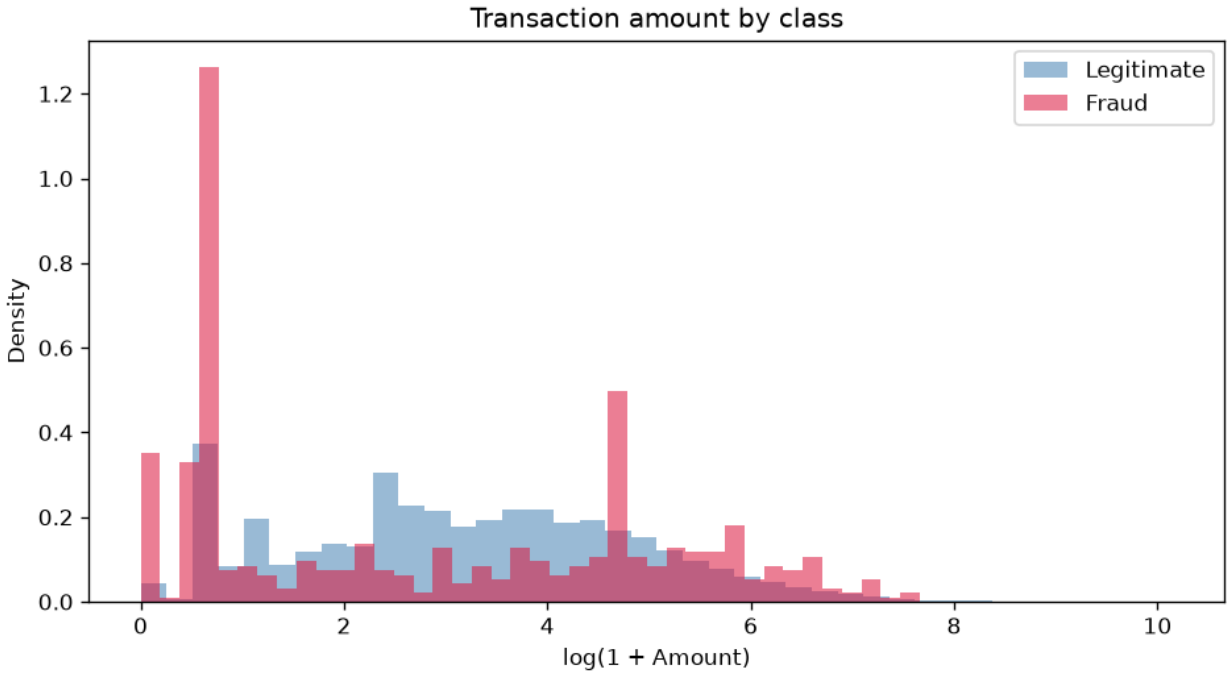
Problem 1A. What is in the file?

Each row is one transaction. Time is seconds since the first payment. V1 to V28 are principal components of the original confidential columns, so they are already numeric and roughly scaled. Amount is the money. Class is 0 for a normal payment and 1 for fraud. There are no missing values.

Fraud amounts are not simply 'the big ones'. Mean fraud amount is $122, mean legitimate amount is $88, but the median fraud is only $9.25. A rule like 'flag large payments' would miss most frauds.



Left: fraud is a sliver, 0.17%. Right: the same counts on a log scale, so the 492 frauds are visible.



Both classes spread across small and large amounts. Amount alone does not separate them.

Problem 1B. Why imbalance breaks accuracy

Predict every payment as legitimate and you are right 284,315 / 284,807 = 99.827% of the time. Recall on fraud is 0%. The loss of a normal model is the sum of mistakes. Almost all of those terms come from legitimate rows, so the easiest way to reduce the loss is to say 'not fraud' everywhere.

The training fix is to change only the training mix. The test set stays at the real 0.17% fraud rate. A resampled test set would be a fake exam.

Stratified 80/20 split, random_state=42: 227,845 train rows (394 frauds) and 56,962 test rows (98 frauds). Time and Amount are on different scales from V1–V28, so StandardScaler is fit on the training columns only.

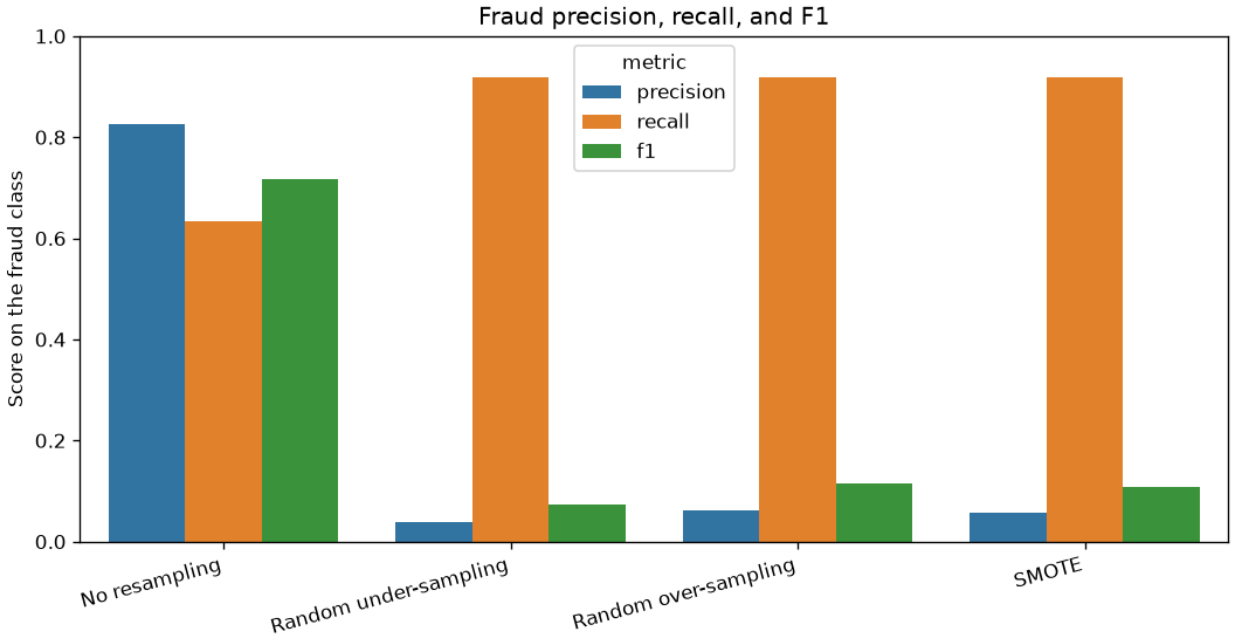
Say this: Accuracy of 99.8% can mean the model caught nothing. For fraud I report precision, recall, and F1 on the fraud class.

Problem 1C. Three ways to rebalance the training set

- Random under-sampling deletes legitimate training rows until 394 frauds and 394 legitimate rows remain (788 rows). Fast, but most normal-payment patterns are thrown away.
- Random over-sampling copies the 394 frauds until there are 227,451 of them. The training set becomes 454,902 rows. Nothing new is invented; the same frauds are repeated.
- SMOTE invents a fraud on the line between a real fraud and one of its nearest fraud neighbours, until the counts match. Same size as over-sampling, 454,902 rows.

The same logistic regression is trained four times. The only change is the training mix. Scores below are on the original test set, fraud class.

Training mix	Precision	Recall	F1
No resampling	0.827	0.633	0.717
Under-sampling	0.038	0.918	0.074
Over-sampling	0.061	0.918	0.115
SMOTE	0.058	0.918	0.109



Sampling lifts recall from 0.63 to 0.92 and knocks precision down near 0.05. F1 follows precision.

Problem 1D. What to conclude

Which method worked best?

By fraud F1, no resampling wins (0.717). It catches 63% of the 98 test frauds and 83% of its alarms are real. All three samplers catch 92% of frauds, but only about 4 to 6 of every 100 alarms are real, so F1 falls below 0.12.

Among the samplers, random over-sampling is slightly ahead of SMOTE, and under-sampling is last, because it also deletes legitimate examples. SMOTE did not beat plain copying: 394 frauds in 30 dimensions is a thin cloud, so a synthetic neighbour is not much new information. Balancing all the way to 227,000 fake frauds also makes the model too willing to shout 'fraud'.

If the bank's real goal is 'miss almost no fraud', a sampler is still useful, but the 0.5 cutoff should then be raised until the false alarms are bearable. F1 at the default cutoff is the wrong only score for that goal.

Which metrics matter?

- Recall: a missed fraud costs money. This is the first number a fraud team asks for.
- Precision: every false alarm is a blocked card and a phone call.
- F1: one number that is bad if either of those is bad.
- Accuracy: ignore it as a headline. 99.83% is the score of doing nothing.

How a hypothesis test helps choose features

Mann-Whitney U asks, for each column, whether fraud values tend to sit higher or lower than legitimate values. It does not assume a bell curve, which matters because Amount is skewed. The test is run on the training set only.

With 200,000 rows, 29 of 30 columns have $p < 0.05$. A tiny shift becomes 'significant' just because the sample is huge. The useful number is the effect size (rank-biserial r, from -1 to 1). Near 0 means the classes are mixed together on that column.

Feature	Effect r	Reading
V14	-0.896	fraud tends to be much lower
V4	+0.884	fraud tends to be much higher
V12	-0.865	fraud tends to be much lower
V10	-0.831	fraud tends to be much lower
V11	+0.825	fraud tends to be much higher

Keep the columns with large |r|. Do not rank columns by p-value. A column with r near 0 can be dropped even if its p-value is tiny.



Red bars: fraud is higher. Blue bars: fraud is lower. Longer bars are the features worth keeping.

Say this: I do not trust accuracy here. Sampling raises recall to 92% and destroys precision. V14 and V4 separate the classes; the p-value does not, because almost every p-value is tiny.

Problem 2. SMS spam

5,572 messages: 4,825 ham and 747 spam. Cleaning is three steps. Lowercase, so Free and free match. Strip punctuation, so free! and free match. Drop English stop words (the, a, to), which show up in both classes.

CountVectorizer builds the vocabulary from the training messages only (6,964 words), then turns each message into a row of numbers. binary=True makes that row 0/1 (word absent or present) for Bernoulli NB. The default keeps the count for Multinomial NB. The split is a stratified 80/20: 4,457 train, 1,115 test. alpha=1 is Laplace smoothing, so a word never seen in spam does not force the whole spam probability to zero.

Model	Accuracy	Precision	Recall	F1
Bernoulli NB	96.59%	0.991	0.752	0.855
Multinomial NB	97.76%	0.949	0.879	0.913

Multinomial NB is better on spam F1 (0.913 vs 0.855). It catches more spam (recall 0.88 vs 0.75). Bernoulli is slightly more precise.

Bernoulli NB treats each word as a yes/no event. Writing 'free' five times is the same evidence as writing it once. That fits a binary bag of words, and it fits short SMS where presence matters more than repetition. Multinomial NB treats the message as a bag of tokens and uses the count. A repeated spam word is repeated evidence. That fits raw counts, or TF-IDF, which is a weighted count. On SMS most counts are already 0 or 1, so the models land close. Multinomial still wins when a spam word is repeated.

Say this: Multinomial NB on word counts wins, F1 0.91. Bernoulli only records whether each word appears, so it misses the extra evidence in a repeated spam word.

Assignment 7, explained

Abalone age groups: logistic regression, PCA, and SVM

A plain-language walkthrough of both parts,
with the numbers from the solution notebook.

SVNIT · Department of Artificial Intelligence
Machine Learning · u24ai028

What the file is, and why Rings becomes a class

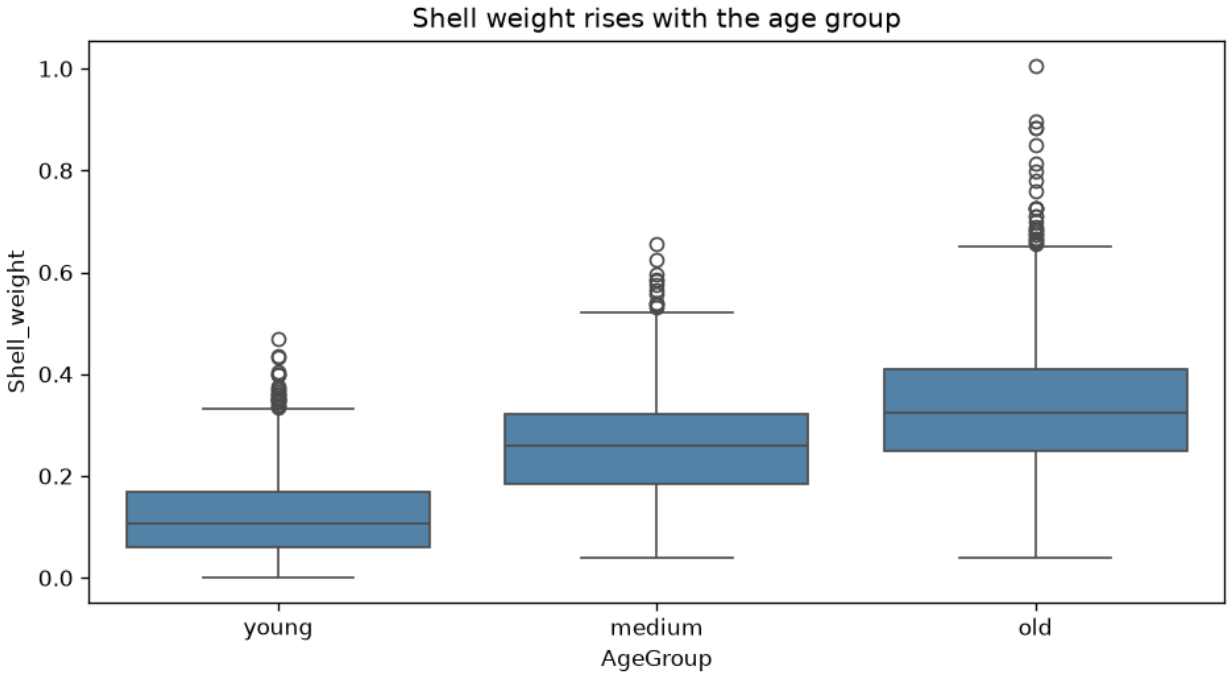
UCI Abalone has 4,177 animals and no header. The columns, in order, are Sex (M, F, or I for infant), Length, Diameter, Height, Whole weight, Shucked weight, Viscera weight, Shell weight, and Rings.

Rings is a count. Adding 1.5 is a rough age in years. Logistic regression and SVM, as used in this lab, predict a category, so the count is cut into three groups of similar size before any model is trained:

- young: 8 rings or fewer (1,407 animals)
- medium: 9 or 10 rings (1,323)
- old: 11 rings or more (1,447)

Those cuts sit near the quartiles of this file. Rings is not given to the model as an input. If it were, the label would be a copy of a feature and the accuracy would be meaningless.

Sex is text. One-hot encoding makes dummy columns. One of the three is dropped, because M + F + I would always equal 1 and would hand PCA a column with no information. The remaining inputs are then standardized: zero mean, unit variance, using the training rows only.



Older groups have heavier shells. Size and age move together, which is why the first principal component will be 'overall size'.

Say this: I turned Rings into young / medium / old, and I did not feed Rings back in as a feature.

Part A. Logistic regression on the scaled features

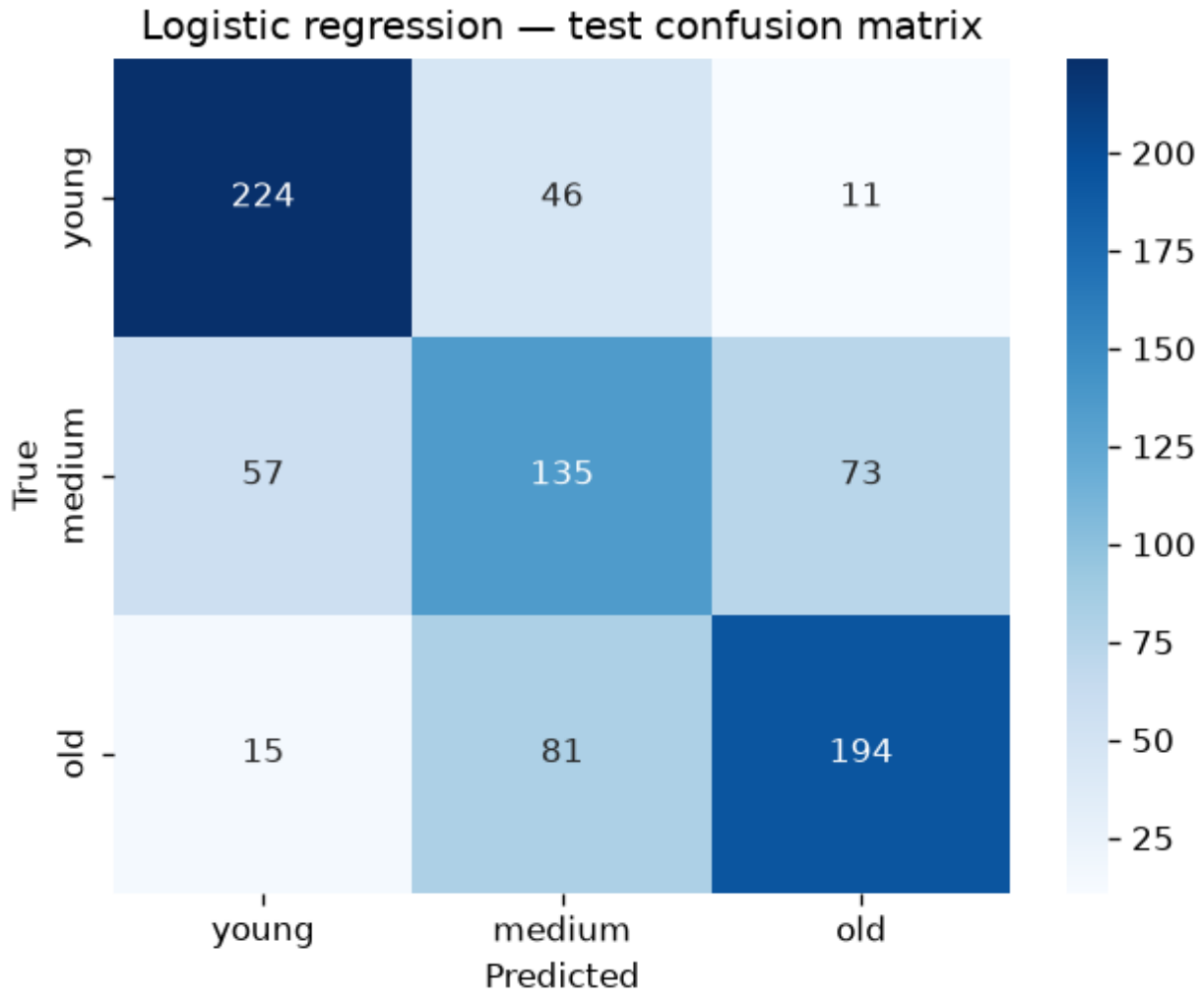
Same split for every model later: 80/20, stratified, random_state=42. That is 3,341 train and 836 test. The scaled matrix has 9 columns: 7 measurements plus Sex_I and Sex_M (female is the dropped baseline).

With three classes, logistic regression learns one score per class and softmax turns the scores into probabilities that add to 1. Current sklearn does this automatically. The old multi_class='multinomial' argument has been removed.

Test accuracy is 66.15%. Young and old are easier. Medium is the overlap: precision and recall are both about 0.51. A medium animal is often called young or old, which is expected because 9–10

rings sits between the other two groups.

True class	Precision	Recall	F1	Support
young	0.757	0.797	0.776	281
medium	0.515	0.509	0.512	265
old	0.698	0.669	0.683	290



Most mistakes are one group away. Young is rarely called old, and the other way around.

Part A. PCA, then logistic regression again

PCA rotates the 9 standardized columns into new axes that are uncorrelated. The first axis is the direction where the animals differ most. explained_variance_ratio_ is the share of variance on each axis:

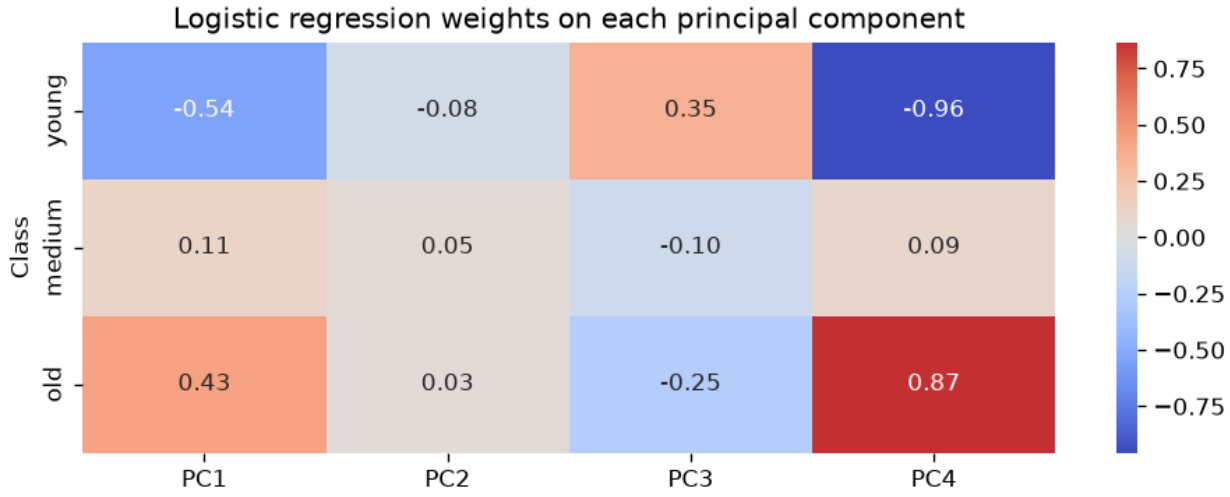
Component	Share	Cumulative
PC1	76.75%	76.75%
PC2	12.81%	89.57%
PC3	4.45%	94.02%
PC4	2.33%	96.35%

Length, diameter, and the weights are almost the same fact ('how big is it'), so PC1 alone holds 77% of the variance. The rule used here: keep the smallest number of components that still hold 95% of

the variance. That is 4 of 9. The other five are dropped before the second logistic regression.

The heatmap is logit_pca.coef_. Rows are classes, columns are PC1–PC4. A positive cell means that component pushes the prediction toward that class. Young has a negative PC1 weight (-0.54) and old has a positive one (+0.43): the big 'size' direction points from young toward old. Medium's weights are all small, which matches it being the class the model is unsure about.

Test accuracy on the 4 components falls to 59.69%. The dropped components still carried some age signal. PCA here buys a smaller input, and it costs about 6 percentage points.



PC1 separates young (negative) from old (positive). Medium barely uses any component.

Say this: Four components keep 96% of the variance. PC1 is size. Accuracy drops from 66% to 60% because some age signal lived in the components we dropped.

Part B. SVM, and the four-way comparison

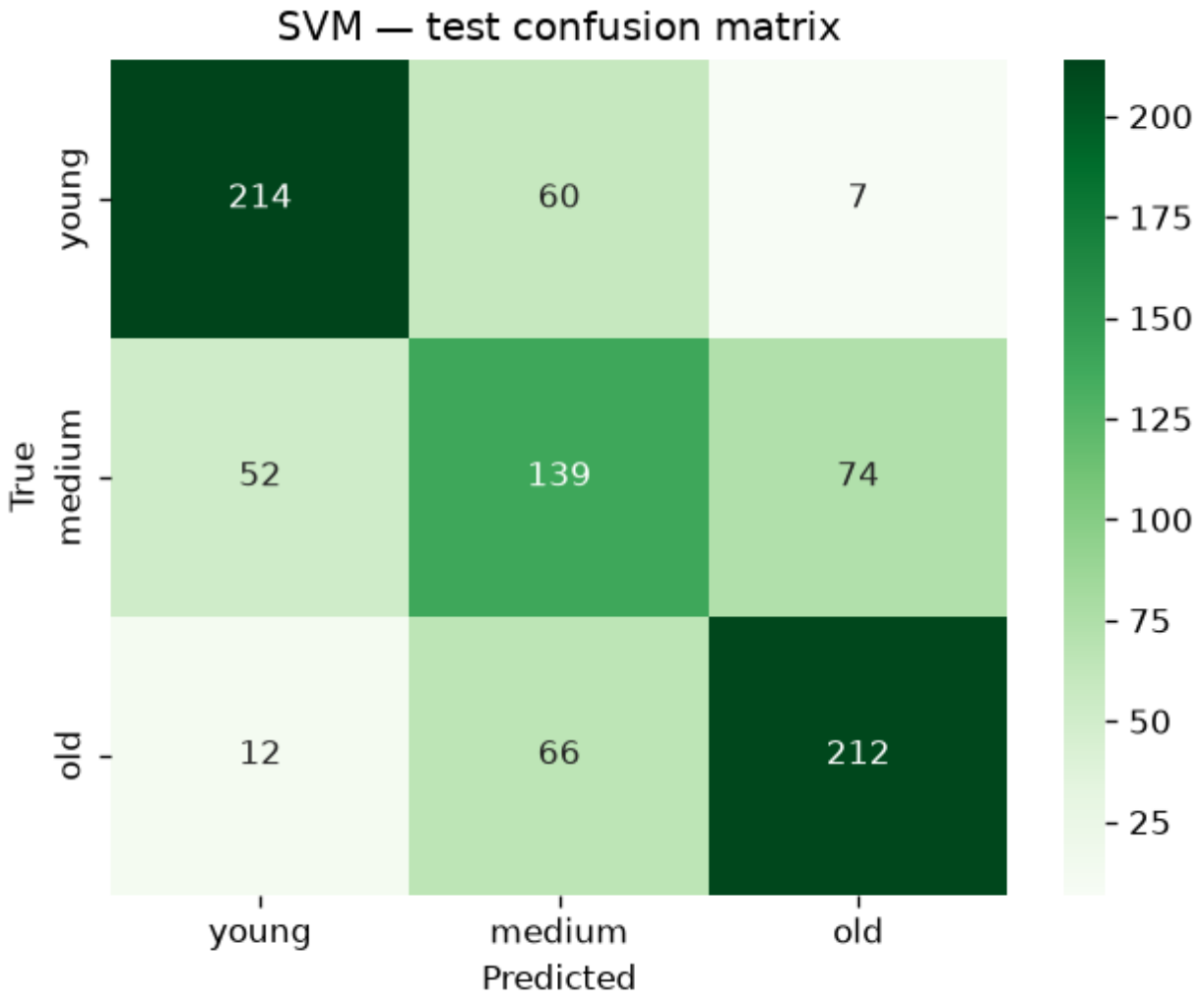
An SVM looks for the widest gap between classes. kernel='rbf' lets the boundary curve. C=1.0 is the penalty for points on the wrong side of that gap. gamma='scale' sets the width of the Gaussian from the data. The same training rows, the same scaler, and the same 4 principal components are reused, so the comparison is fair.

Model	Features	Test accuracy
Logistic regression	9 standardized columns	66.15%
Logistic regression	first 4 PCs	59.69%
SVM, RBF kernel	9 standardized columns	67.58%
SVM, RBF kernel	first 4 PCs	62.32%

The RBF SVM on the original standardized columns is the best of the four, at 67.58%. It only barely beats logistic regression (66.15%). The classes overlap in size, so a curved boundary helps a little, not a lot. PCA lowers both models by about 6 points. Use PCA here when you want fewer, uncorrelated inputs. Use the full standardized columns when you want the best test accuracy.



SVM on the full scaled features is the best bar. Both PCA models sit lower.



Same pattern as logistic regression: young and old are recognised, medium is the confused class.

Say this: Best test accuracy is the RBF SVM on the 9 scaled columns, 67.6%. PCA to 4 components keeps 96% of the variance and costs about 6 points of accuracy.

Assignment 8, explained

A neural network that reads handwritten digits

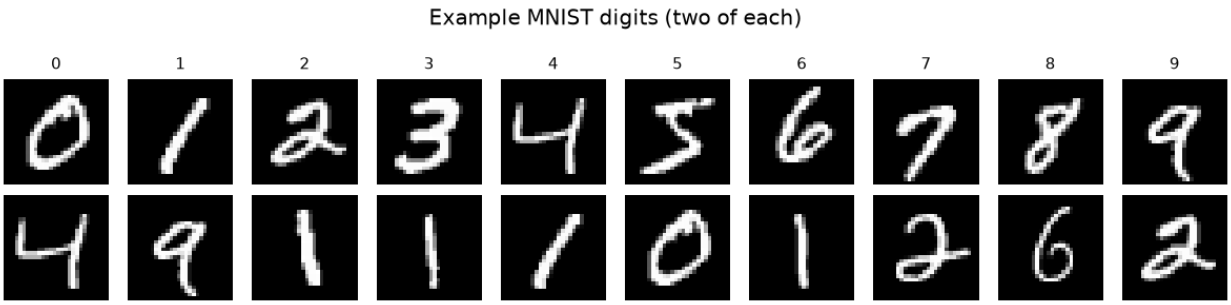
A plain-language walkthrough of the MNIST network,
with the numbers from the solution notebook.

SVNIT · Department of Artificial Intelligence
Machine Learning · u24ai028

What you are asked to build

MNIST is 70,000 pictures of handwritten digits, already split into 60,000 for training and 10,000 for testing. Each picture is 28 by 28 pixels, grayscale, and the label is the digit 0–9.

The model is a feed-forward network: an input layer, hidden layers, and a softmax output. It is trained with cross-entropy loss and Adam. This lab environment does not have TensorFlow or PyTorch, so the notebook uses sklearn's MLPClassifier. That class is the same kind of network: 784 inputs, ReLU hidden units, a 10-way softmax, cross-entropy, and solver='adam'.



Two examples of each digit. The same digit is written differently every time, which is why a straight classifier is not enough.

Preprocessing

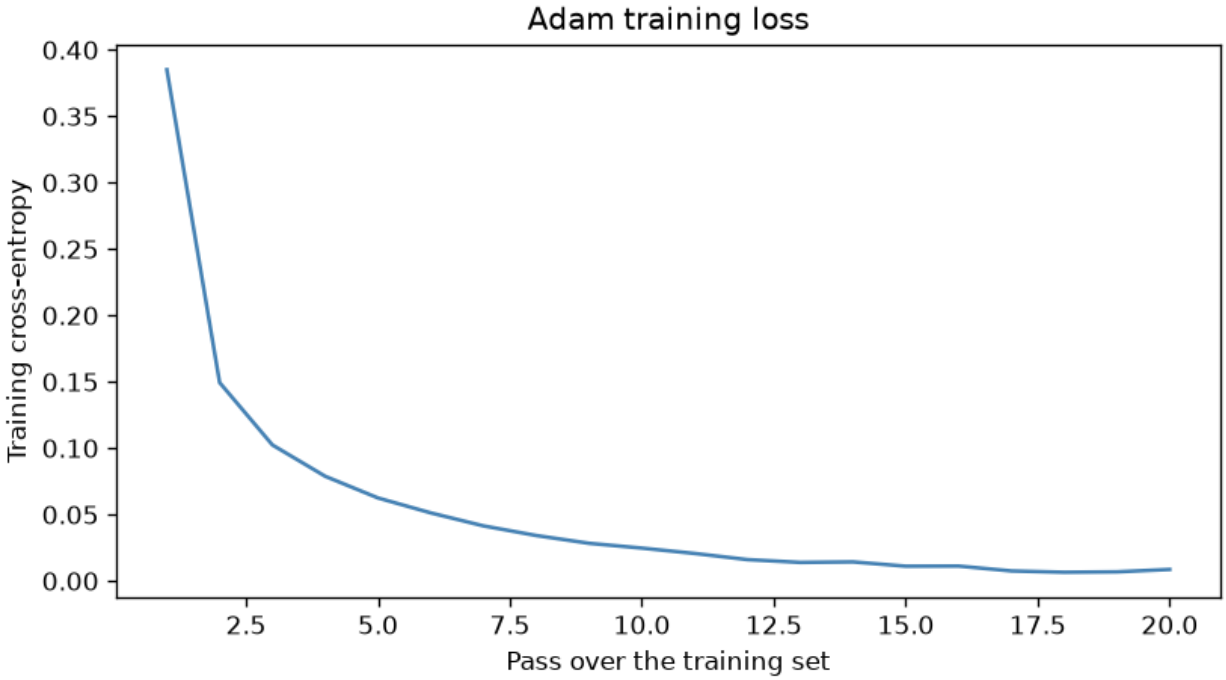
- Pixels arrive as integers from 0 to 255. Divide by 255 so every input is between 0 and 1. Large inputs make the first weight updates too violent.
- Flatten each 28×28 picture into a vector of 784 numbers. An ordinary neural net has no notion of 'the pixel above'. A convolutional net would keep the grid; this assignment asks for an ANN.
- The official 60,000 / 10,000 split is kept. Reshuffling it would make the test score incomparable with published MNIST numbers.

The ten digits are roughly balanced in the training set (about 5,400 to 6,700 images each), so accuracy is not the trap it was on the fraud data.

The network, layer by layer

- Input, 784 units. One unit per pixel. This layer stores the image. It does not learn weights of its own.
- Hidden layer, 128 units. Each unit computes ReLU(w · x + b), and ReLU is max(0, z). Stacking these lets the net bend, which logistic regression cannot do.
- Hidden layer, 64 units. A second ReLU layer, reading the 128 units rather than the raw pixels.
- Output, 10 units, then softmax. Softmax turns the 10 scores into probabilities that add up to 1. The predicted digit is the largest probability.

Cross-entropy is the loss. If the true digit is 7 and the network says 7 with probability 0.9, the loss is small. If it says 7 with probability 0.01, the loss is large. Adam updates each weight with its own step size, based on how noisy that weight's gradient has been. The starting step size used here is 0.001. The network sees the training set 20 times. Training cross-entropy ends at 0.0086.



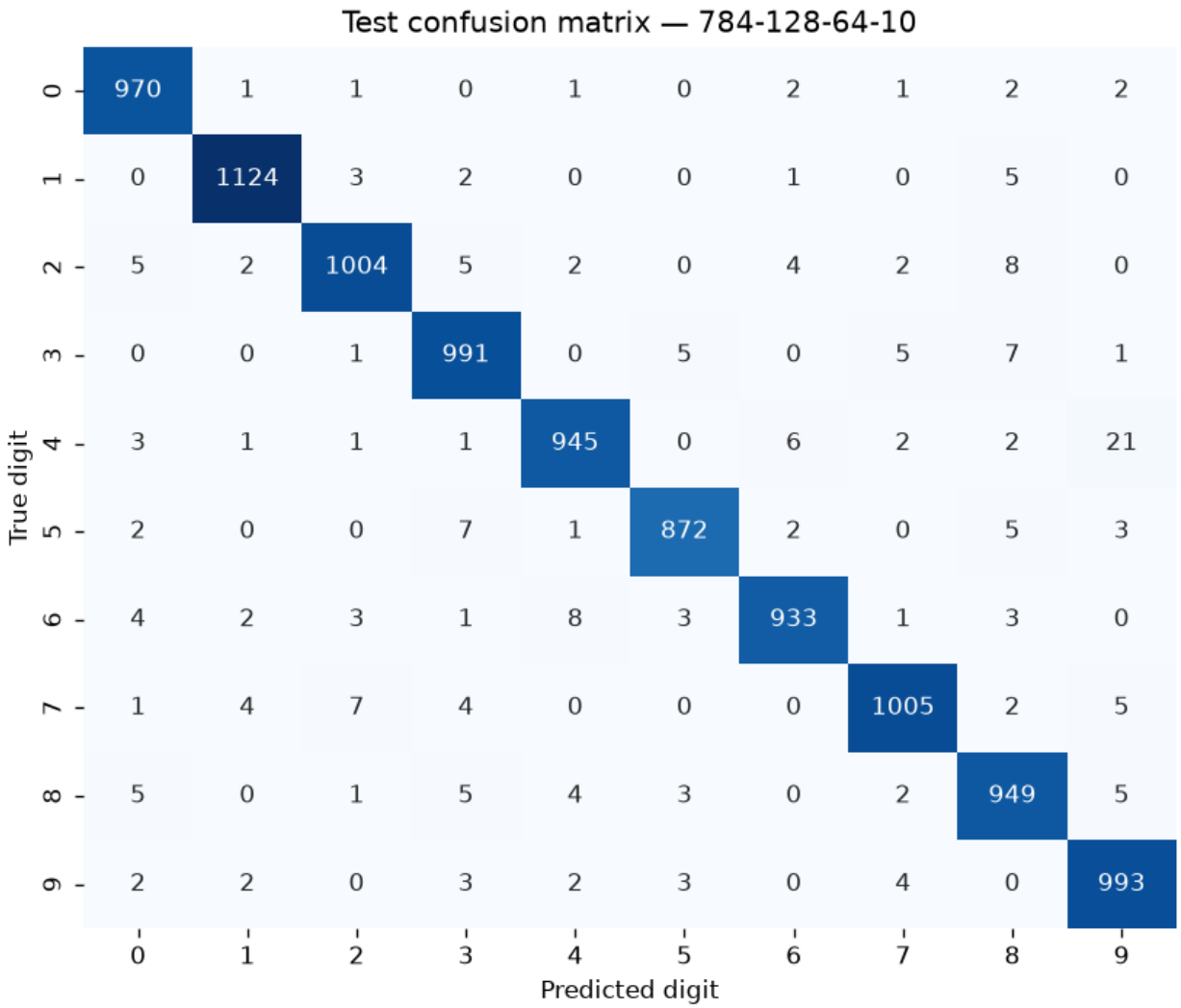
The loss falls quickly and then flattens. Twenty passes are enough for this size of network.

Test scores

On the 10,000 images the network never trained on:

Metric	Score	Meaning
Accuracy	97.86%	fraction of digits read correctly
Precision	0.979	macro: each digit weighted the same
Recall	0.978	macro: each digit weighted the same

Digit 1 is the easiest (precision and recall both about 0.99). Digits 4, 8, and 9 are the ones it mixes: a 4 can look like a 9, and an 8 can look like a 3 or a 5. The confusion matrix is almost diagonal, which is what a good digit reader looks like.



Rows are the true digit, columns are the prediction. Off-diagonal cells are the mistakes.

Say this: The 784-128-64-10 network reads 97.86% of the test digits. Precision and recall are essentially the same number because the classes are balanced.

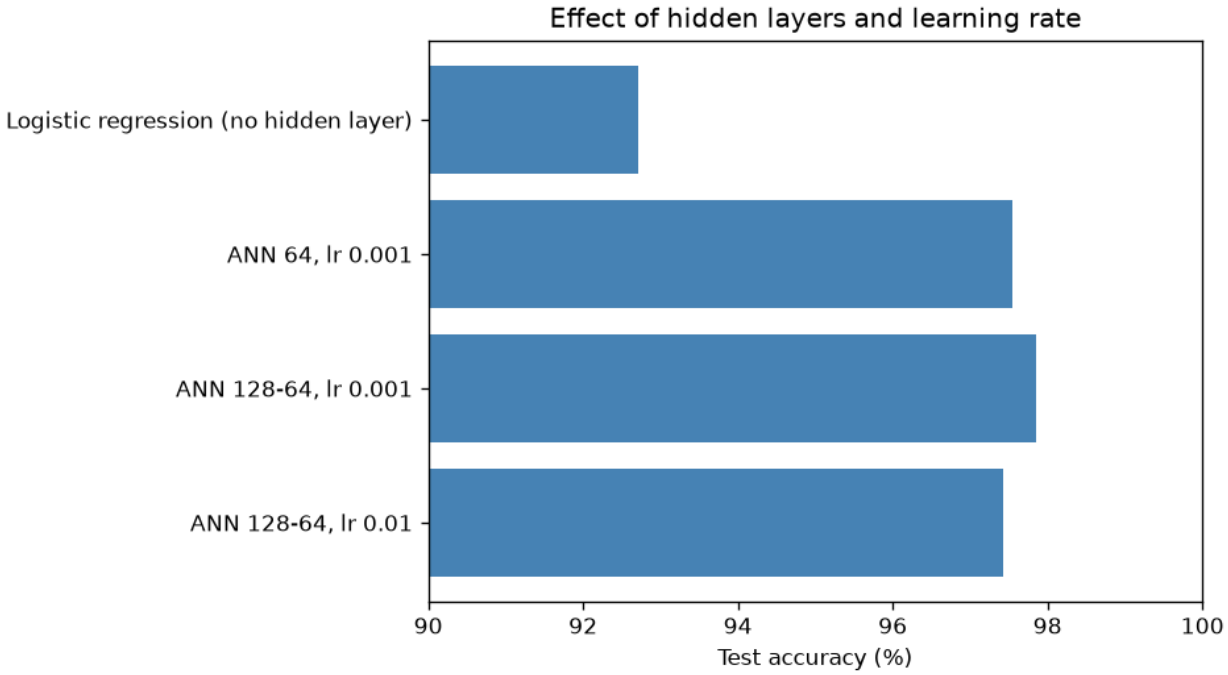
Hyperparameters, against a plain benchmark

One change at a time. Logistic regression is the same softmax with no hidden layer, so it shows what the hidden layers are worth.

Model	Accuracy	Precision	Recall
Logistic regression	92.71%	0.926	0.926
ANN, 64 units, lr 0.001	97.54%	0.975	0.975
ANN, 128-64, lr 0.001	97.86%	0.979	0.978
ANN, 128-64, lr 0.01	97.42%	0.974	0.974

- Adding even one hidden layer jumps accuracy from 92.7% to 97.5%. A digit is not a straight cut through pixel space.
- A second hidden layer adds a bit more, 97.54% to 97.86%. Extra capacity helps, then the gains get small.
- Raising Adam's step from 0.001 to 0.01 lowers accuracy to 97.42%. The network still learns, but the steps are too coarse and it settles in a worse place.

Published context, not trained in this notebook: a linear classifier on MNIST is usually about 92%, which matches the logistic regression here. A fully connected network of this size is usually 97–98%, which matches this ANN. A convolutional network, which knows that neighbouring pixels form strokes, is usually above 99%. This network cannot use that, because the picture was flattened.



The axis starts at 90% so the gap between logistic regression and the networks is visible.

Say this: Hidden layers are worth about 5 points over logistic regression. Two layers and a learning rate of 0.001 is the best of the four settings I tried, at 97.86%.